Anti-porous architecture: a unified design principle for CQRS + Actor + Event-Sourcing systems

Alvaro Rivera

2026-05-18

Abstract

This paper is a design theory contribution in the sense of Hevner, March, Park, and Ram (2004): it introduces a new design construct, derives design principles for its application, and presents an instantiation that demonstrates the construct is realizable in production. Four bodies of literature — relational database theory, domain-driven design, REST API contract design, and Event Sourcing — independently document a recurring representational defect: schemas that mix unrelated concepts, fields populated for some rows but not others, contracts overloading heterogeneous use cases behind optional parameters, event payloads recording what was sent rather than what was used. Each tradition names the defect locally and prescribes a local remedy. This paper argues that the four are surface manifestations of a single structural defect, which we call *porosity*: the shape of a representation is dictated by what fits in the storage substrate rather than by the operations the representation describes. We further argue that *anti-porosity* — the principled rejection of porosity simultaneously across the three layers in which it manifests (domain, endpoint, persistence) — inverts the underlying assumption (domain models shaped for verbs rather than for storage), and that the conditions under which a system satisfies it are independent of any specific framework. A system descending from a 2005 wrapper, refined across multiple projects and ported from Java to .NET, has run continuously in production since 2018 satisfying these conditions; that system, Puppeteer, is presented as one realization, not as the foundation of the claim.

Anti-porous architecture

TL;DR

Architectures that shape their domain models to fit a storage substrate produce *porous* designs under structural pressure: rich object graphs — recursive, heterogeneous, with different node and edge

types — do not survive a relational schema, so domain classes either fragment across joined tables that return empty cells or collapse into wide tables of redundant primitives; endpoint DTOs accumulate optional fields to serve heterogeneous use groups behind a single contract; and persistence layers record *what state exists* without recording *how it came to be*.

This paper argues that these are surface manifestations of a single structural defect — *porosity* — that four bodies of literature (database theory, domain-driven design, REST API design, Event Sourcing) have each diagnosed locally without recognizing as one. We name the unified rejection *anti-porosity*, and we identify three simultaneous conditions a CQRS + Actor Model + Event Sourcing combination must satisfy to sustain it. A system satisfying these conditions exists; it is presented in §4 as one realization, not as the foundation of the claim. **Porosity is not a database problem, nor an API problem, nor an event-sourcing problem; it is a representational sparsity problem described independently in graph theory, information theory, database normalization, and storage engine theory.**

Claims this paper makes

1. **Porosity has three distinct surface manifestations.**
   - *Domain*: a class hierarchy whose shape is dictated by what fits in the storage substrate rather than by the operations the domain supports — losing references, recursion, and heterogeneous typing along the way. Two visible patterns: over-normalized schemas, where classes fragment across many tables joined by type and queries return rows of mostly-empty cells; and under-normalized schemas, where classes collapse into wide tables with naming ambiguity and primitive-level redundancy because complex objects must be flattened to fit a row.
   - *Endpoint*: a single API contract serving heterogeneous use groups via optional fields, with no contractual indication of which fields belong to which group.
   - *Persistence*: representations that capture *what state exists* without capturing *how it came to be* — a row of weights, but not the operations that produced them. Rich object graphs (recursive, heterogeneous, with different node and edge types) either fragment or flatten as above; document stores partially relieve this for tree-shaped data but degrade once recursion or cross-references appear; and naïve event sourcing recreates the defect at a different scale by serializing every input parameter regardless of which the execution consumed.
2. **The three are not independent problems.** Each is a downstream ef-

fect of the same upstream decision — modeling the domain *for* the storage substrate (primarily relational, but the same pattern partially recurs with document stores when recursion or cross-references are present). Once that decision is in place, every local fix recreates the pressure in a different layer.

3. **A combined CQRS + Actor Model + Event Sourcing implementation can reject all three porosities using a small, shared set of primitives.**
    - *Domain*: one class per role, with arbitrary references — including recursion and heterogeneous typing — permitted in the in-memory model. The persistence substrate no longer constrains the domain shape.
    - *Endpoint*: DTOs with filterable optional parameters; the same contract serves heterogeneous use groups without polluting the journal.
    - *Persistence*: dense journals of *verbs* rather than state. Each entry records a high-level operation designed by the programmer and exactly the inputs that operation consumed — not the edges of the object graph, not the input parameter set as received. The state, however rich, is reconstructed deterministically by replay.
    - The three mechanisms operationalize the same notion (*"what this operation actually used"*) at three layers.
4. **The principle is not theoretical.** A system satisfying the three conditions of Claim 3 exists. It descends from a 2005 wrapper through a sequence of internal projects, was ported from Java to .NET, and has run continuously in production since 2018, currently in the core of eCommerce and payments platforms — payment hubs, account-balance ledgers, KYC pipelines, customer-facing storefronts and experiences, and payment-processor integrations. The system is the Puppeteer framework; §4 describes how it realizes each condition (with §4.0 covering the historical genealogy). Code references throughout this paper (Appendix A) point to the public repositories; quantitative empirical results are deferred to a forthcoming case-study white paper.
5. **The contribution is the cross-literature identification, not the implementation.** Four bodies of work — relational database theory, domain-driven design, REST API contract design, and Event Sourcing — have each documented surface manifestations of porosity in their own vocabularies, without recognizing the underlying unity. CQRS, the Actor Model, and Event Sourcing are individually well-known constituent patterns; what we claim as novel is neither the patterns nor the detection of any single porosity, but the observation that the four traditions converge on a single defect, that *anti-porosity* names its unified rejection, and that the rejection is implementable rather than merely conceivable. Other actor frameworks could realize the same conditions through different mechanisms (§7); the conceptual claim is independent of the realization.

6. **CQRS + Actor + Event Sourcing is interesting here for density preservation, not for separation of concerns.** These three patterns are conventionally introduced as separations: reads from writes (CQRS), concurrent units (Actor Model), events from projections (Event Sourcing). When applied together as a single discipline, they constitute a mechanism that preserves density across the three representational layers in which porosity manifests. This interpretation is available only once porosity is named as a unified phenomenon across substrates.

When NOT to use this approach

The principle of anti-porosity has limits, and the implementation pattern that realizes it has narrower limits still. We name two regimes in which the implementation is not the right fit. In each case, anti-porosity may continue to apply as a diagnostic — porosity remains a real pathology — but the CQRS + Actor + Event Sourcing realization, as instantiated in Puppeteer, imposes costs that may not be repaid.

A. Domains where the actor cannot achieve state locality

The actor model with journal-based persistence assumes that the working set of events required to rehydrate the actor's current state can be bounded. This holds when the actor's automaton passes through cyclic states — created, modifiable, finalized, archived — such that, at some point in the cycle, prior events become safely evictable without altering the current state. Domains where the actor's lifecycle has no such evictable point produce *unbounded rehydration*: every load replays a history that grows without ceiling.

Consider an airline flight. From the moment a schedule is published, an actor we may call `FlightOperation` accumulates events: the first booking, subsequent bookings, seat assignments, gate changes, boarding events, departure, landing. Throughout this sequence the state is genuinely active — answering *"is the flight bookable?"*, *"who is aboard?"*, *"has it departed?"* requires recent history. But once the flight has landed and reconciliation completes, the active state collapses: the flight no longer changes, and the remaining consumers are analytical (capacity-utilization reports, for instance) and can be served from periodic projections. At that point the entire history of bookings and gate changes can be marked evicted without compromising correctness. The actor has reached locality: its working set is bounded.

Compare this to an `Airline` actor that consolidates every flight, every passenger, every booking the airline has ever handled. Such an actor never reaches a "done" state. Its working set grows with operational history, and each rehydration replays material that has no bearing on any decision currently being made. The two designs share the same business domain; they differ in whether the actor's responsibility has a natural end.

The structural analogy with virtual memory is exact. In an operating system,

a process exhibits *locality of reference* when its accessed pages stay within a bounded working set; the OS evicts pages outside that set without affecting correctness. When locality is lost — when references span the full address space uniformly — the system enters *thrashing*: pages are reloaded as fast as they are evicted, and useful work approaches zero. The actor model with a journal exhibits the same structural property: events take the role of pages, the actor's required history takes the role of the working set, and skips (event eviction) take the role of page replacement.

Virtual Memory	Actor + Journal
Working set bounded	Non-skipped events bounded
Locality of reference	Locality of automaton state
Page eviction	Skip / event eviction
Page size	Actor granularity
Thrashing	Unbounded rehydration

The locality property of an actor is, to a substantial degree, downstream of its naming. The word *actor* evokes *action*: a `Packer`, a `Dispatcher`, a `FlightOperation` — names that denote bounded responsibility with a natural end. Names that denote aggregations (*Inventory*, *Catalog*, *Customer*) imply unbounded responsibility, with no point at which prior events become evictable. A noun-aggregator actor is the structural equivalent of an OOP god class: state accumulates because nothing about the name suggests when it should stop. The same observation applies one level further down, to the partitioning of instances: an actor scoped per-country rather than per-shipment reproduces the problem even when the type is well-named. Granularity and partitioning are not separate decisions from naming; they are the same decision expressed at different levels.

Microsoft Orleans makes this granularity choice explicit in the very name of its activations — *grains* — emphasizing each as a small, self-contained unit. The naming heuristic developed here is consistent with that view and offers a verifiable test: if the proposed actor name is a noun-aggregator rather than a verb-doer, the design will likely fail at locality.

In typical production deployments, the cost of locality failure is partly amortized. Continuous-running datacenters confine rehydration to rare events — process restart, hardware failover — rather than steady-state operation; zero-downtime release patterns (developed in Paper 3) hide cold-start latency behind warm replicas during deployment; and journal storage is operationally cheap: it is economically negligible at relevant scales, and its access pattern — sequential append, with occasional sequential reads at rehydration — does not impose the random-I/O performance demands that drive RDBMS disk specifications. A Puppeteer deployment is largely indifferent to disk speed, where a comparable RDBMS workload is bound by it. Locality is therefore best understood as a

structural assumption of the model rather than a property whose violation is immediately catastrophic. The "when not to use" case bites when locality fails **and** the amortizing properties above do not hold — for instance, single-instance systems with frequent restarts, or workloads where actor cardinality is so high that eviction-and-reload occurs as part of normal operation.

The mechanism by which Puppeteer evicts journal events — the *skip* — is treated in detail in a companion paper (Paper 4, on zero-downtime deployment via journal-based state handoff). For present purposes it suffices to note that skips presuppose locality: the framework can implement skips, but it cannot manufacture locality where the actor's responsibility is unbounded. Locality is a property of the design, not of the framework.

B. Domains where the actor's verbs cannot be kept fast

Each actor processes its mailbox serially: only one verb executes at a time on a given actor instance. The framework sustains thousands of requests per second per actor, at peak, on the assumption that every verb completes within a few milliseconds. A verb that blocks for hundreds of milliseconds — synchronous I/O to a slow service, an expensive query, a costly computation — does not merely slow itself; it blocks every subsequent verb in the mailbox, and ingest throughput collapses. The single-thread invariant of the actor turns from a correctness guarantee into a throughput cap.

Two patterns sustain the fast-verb invariant in practice. **I/O hoisting**: the caller resolves external dependencies — third-party calls, slow lookups, expensive joins — before invoking `perform`, and passes the resolved values as parameters. The actor receives data; it does not fetch. **Saga-phased I/O**: when the workflow genuinely requires interleaved external calls, the work is decomposed into a Saga. The actor advances one phase, releases, an external coordinator performs the I/O, and the response re-enters the actor as the next event. The actor never waits inside a single verb.

Where neither pattern applies — synchronous I/O is intrinsic to the verb, or the underlying algorithm is unavoidably slow — the actor's serial processing is structurally incompatible with the workload. In those cases the framework's claim of high per-actor throughput cannot be honored.

In both regimes, the principle of anti-porosity may continue to apply as a diagnostic; what fails is the realization, not the principle.

---

1. Introduction: the invisible cost of porous designs

This paper makes a design theory contribution. It identifies and names a structural defect that prior literature has documented separately without recognizing as one (the construct, *porosity*); derives the design principles required to reject it across the three layers in which it manifests (*anti-porosity*); and presents an

instantiation — a system whose underlying observations date to a 2005 prototype and whose current form has been in continuous production use since 2018 — as confirmation that the construct is realizable. The contribution is conceptual; the instantiation is the existence proof of realizability, not the substance of the claim. The genre is the one Hevner, March, Park, and Ram (2004) name *design science research*: empirical evidence is presented in the form of a working artifact rather than a controlled experiment.

A defect recurs in mainstream software architectures: representations that carry more structure than the operations they describe ever populate. Relational schemas accumulate columns whose values depend on the row's state — required for some, irrelevant for others. API contracts grow optional fields that serve different use groups under one route, with no contractual signal of which fields belong to which case. Event payloads serialize whatever the command DTO carried, regardless of which inputs the operation consumed. Each looks local, and each has a literature that diagnoses it locally: database theory names normalization anomalies and NULL semantics; domain-driven design names anemic and persistence-driven models; REST API design names overloaded contracts and field bloat; Event Sourcing names payload bloat and command-event coupling. Four traditions, four vocabularies, four sets of remedies. None observes that the four describe the same defect.

We call this defect *porosity*: the shape of a representation is dictated by what fits in the storage substrate rather than by the operations the representation describes. Its visible symptom is widespread structural emptiness — fields without values, columns whose meaning depends on the row, queries returning rows with cells that the application never reads. The defect emerges most visibly when complex or polymorphic abstractions are forced into relational tables, and intensifies as the abstractions become richer. The vocabulary for the unified phenomenon has not existed: developers do not say *this design is porous*; they say *this design is fine; the empty fields are normal.* Decades of relational-first modeling have made the trade-offs that produce porosity (over-normalization versus under-normalization, joins versus wide rows, NULLs versus duplicate tables) feel like inherent properties of software construction. They are not. They are the consequence of one architectural decision — that the domain model must be shaped for the database — repeated millions of times. Naming the phenomenon is the first step toward seeing it.

This paper argues that porosity is not inevitable, that the four traditions cited above describe surface manifestations of a single structural defect, and that their independent local remedies are partial solutions to a problem that admits a unified one. We name the unified rejection of porosity *anti-porosity*, identify the conditions under which a CQRS + Actor Model + Event Sourcing combination sustains it across all three layers in which it manifests (domain, endpoint, persistence), and present a system that satisfies those conditions as confirmation that the principle is realizable. The historical genealogy of that system — how it came to satisfy the conditions before they were named — is presented in §4.0,

alongside the mechanisms it uses to do so.

The paper is organized as follows. §2 enumerates three layers of porosity and argues that each is a face of the same defect. §3 introduces anti-porosity as a unified design principle and states the conditions a system must satisfy to realize it. §4 describes the origins and mechanisms by which one such system, Puppeteer, realizes those conditions; the section is descriptive of an instantiation, not constitutive of the principle. §5 reports operational observations drawn from production deployments. §6 addresses anticipated counter-arguments from the principle. §7 places this work in the context of related literature. §8 concludes.

1.1 A formal characterization

Porosity is not a database problem, nor an API problem, nor an event-sourcing problem. It is a representational sparsity problem described independently in graph theory, information theory, database normalization, and storage engine theory.

The four formalisms that establish this characterization are summarized below; their intersection makes the term derivable rather than invented.

Graph theory. Domain models can be formalized as *semantic graphs*: nodes typed by entity class, edges typed by relation, with both nodes and edges potentially heterogeneous in type (Diestel, 2017). Polymorphism corresponds to heterogeneous node typing; recursion corresponds to self-referencing edge structure. The graph-database and Semantic Web literatures (Berners-Lee et al., 2001; Robinson et al., 2015) acknowledge this directly; the relational literature does not.

Information theory. Following Shannon (1948), a representation can be characterized by its *structural capacity* — the bits available to encode states — and its *informational content* — the bits actually used. When structural capacity exceeds informational content, the representation is *sparse*: most of its bits carry no information for any given instance. Empty fields, NULLs, and unused parameters are concrete realizations of representational sparsity in software systems. Unused parameters in a serialized event are, in the strict information-theoretic sense, *structural noise*; preserving only the consumed parameters is a noise-suppression operation, not a stylistic preference.

Storage engines. Each storage substrate exposes a *structural alphabet*: the set of shapes it can natively express (Hellerstein et al., 2007). Relational stores express rows × columns × types; document stores express nested trees of typed values; event stores express ordered logs of typed records; graph stores express labeled directed graphs. The alphabet of a substrate determines what can be written densely; whatever exceeds the alphabet must either be projected, with loss of structure, or padded, with empty cells. The relational case is particularly sharp: the SQL row is a projection onto a homogeneous vector space — every row a tuple of fixed dimension and uniform schema — and any semantic

structure that does not fit (heterogeneous types, recursion, sparse attributes) is paid for in NULLs, joins, or schema duplication.

Database normalization theory. Codd's relational model (Codd, 1970), refined through successive normal forms (Codd, 1971; Fagin, 1977), is the canonical historical formalism for representing data in the relational alphabet. Normalization is prescribed to eliminate update, insert, and delete anomalies. The procedures that resolve these anomalies — splitting wide tables, introducing join columns, accepting NULLs in optional attributes — themselves produce the representational sparsity that, decades later, would be accepted in practice as the cost of doing software.

Synthesis. The four formalisms converge on a single definition:

> *Porosity is the architectural manifestation of representational sparsity that arises when a semantic graph is projected onto substrates whose structural alphabet exceeds the information actually required by operations.*

The dual:

> *Density preservation occurs when representations encode only the information actually consumed by operations, keeping structural capacity proportional to semantic content.*

Each phrase in these definitions is borrowed from one of the four formalisms; the contribution is the act of synthesis — observing that the formalisms converge on a defect that none of them names with the generality needed to see across substrates.

2. Three faces of porosity

2.1 Porosity in the domain layer

In object-oriented modeling, the canonical pattern for representing entities that pass through distinct lifecycle states is inheritance: each state is a subtype, with its own fields, invariants, and methods. A purchase order, for example, naturally decomposes into `Draft`, `Requested`, `Paid`, `Dispatched` — each subtype carrying only the attributes and operations that its state admits. The transition from one state to another is the construction of a new subtype instance, not the mutation of fields on a single class.

The relational model does not support this pattern cleanly. A type hierarchy can be projected onto a relational schema via one of three known idioms — single-table, class-table, or concrete-table inheritance (Fowler, 2002) — each of which produces porosity in a different form. In practice, single-table inheritance dominates: the schema collapses the four subtypes into one wide table with a `status` column and optional fields populated only for some rows. Attributes that belong logically to `Paid` are nullable for rows still in `Draft`; attributes that

belong to `Dispatched` are nullable for everything else. The hierarchy is recovered at read time by inspecting the `status` field and conditionally interpreting the rest. The semantic graph — a clean polymorphic structure — has been projected onto a homogeneous vector space, and the gap between them surfaces as NULLs.

Formally, the relational projection encodes a sum type as a product type. The state hierarchy — `Draft`, `Requested`, `Paid`, `Dispatched`, each variant with its own fields and invariants — is naturally a tagged union: a sum type. The relational schema, lacking native sum-type support, encodes it as a product type — a single tuple of fields that must coexist regardless of variant, with the discriminator demoted to a column. The mismatch is structural. In the information-theoretic vocabulary of §1.1, sparsity — structural symbols present in the representation that carry no information for any specific instance — is the inevitable cost of forcing a sum-type structure through a product-type alphabet.

The choice is not driven by ignorance of the alternative. Developers familiar with object-oriented design recognize that inheritance is the structurally cleaner pattern; they adopt the property-field approach because the substrate makes it the path of least resistance. The other two idioms impose joins by type, schema duplication, and migration overhead severe enough to outweigh the modeling benefit. Porosity here is the signature of a forced compromise — not an absence of OOP wisdom, but a substrate that does not admit it.

The downstream effects extend beyond schema shape. State transitions on the porous table become updates against a row that other transactions may also read or modify; lock contention emerges as a structural cost of the design choice, not as an accident of high traffic. The broader observation — that a domain whose lifecycle was never logically concurrent now pays for transactional concurrency it did not request — is treated separately, as a distinct symptom of persistence-as-source-of-truth, in Paper 5.

2.2 Porosity in the endpoint layer

The endpoint layer's primary contractual artifact is the DTO — a fixed-shape tuple of fields defining what flows between client and server. When an endpoint serves a single, homogeneous use case, the DTO is well-defined and dense: each field participates in the operation's semantics.

When an endpoint serves heterogeneous use groups behind a single route, however, the DTO accumulates optional fields, mirroring the wide table with `status`-conditioned columns described in §2.1. The structural defect is identical; only the layer differs.

From the perspective of type theory, a DTO is typically modeled as a product type: a fixed collection of fields that must coexist. Heterogeneous use groups, by contrast, correspond naturally to a sum type — a tagged union — where each variant carries only the fields relevant to its case. When a product type is

forced to encode what is semantically a sum type, sparsity appears as optional fields and conditional validation.

Consider an endpoint `POST /orders` that serves both a customer self-checkout flow and an administrative batch-insertion flow. The customer flow specifies items, a delivery address, and a payment token; the administrative flow specifies override pricing, source attribution, and an internal correlation ID. A unified DTO carries the union of all fields, validates conditionally based on caller identity or feature flags, and provides no contract-level indication of which fields belong to which use group. Documentation shoulders the burden the schema cannot.

Formally, this is again a projection of heterogeneous semantic variants onto a homogeneous vector space. In terms of information theory, the DTO's structural capacity exceeds the information actually conveyed by any single call. The unused fields constitute representational sparsity: structural symbols present in the representation that carry no information for the specific operation being performed.

The defect manifests symmetrically on the response side. The same DTO that returns an order's full detail to a desktop client returns more than a mobile client requires; clients either over-fetch or fragment the operation into multiple round-trips, neither of which the original fixed-shape schema admits. The DTO, as a homogeneous projection of underlying semantic variants, reproduces at the wire contract level the porosity already observed in relational schemas of §2.1.

Costs accumulate at multiple levels. Validation logic becomes conditional. DTO families proliferate as variants are introduced to handle slightly different use groups. Client code absorbs out-of-band knowledge about which fields apply when. The contract — intended to be the disciplined surface between systems — becomes porous: its meaning depends on contextual knowledge that the contract itself does not encode.

2.3 Porosity in the persistence layer

The persistence layer's job is to durably capture what the system has done. The shape of that capture is determined by the substrate. Three regimes recur in practice — relational stores, document stores, and conventionally implemented event sourcing — and each manifests porosity differently while sharing a common structural cause: the alphabet of the substrate fails to align with the structure of the domain it is asked to record.

Relational persistence captures state, not the operations that produced it. A row records that an order is in `paid` state and stores the amount and payment method, but cannot answer how the order arrived there or what triggered the transition. The state-conditioned fields of §2.1 propagate to persistence: when invariants depend on state — for instance, `amount` is required only when `status = paid` — the schema cannot enforce them. The available alphabet (foreign

keys, nullability, primitive `CHECK`) admits only primitive referential integrity, never the conditional, cross-field, cross-object invariants that emerge naturally from a sum-type domain. Invariant enforcement migrates to code by structural necessity, not by stylistic choice.

Document stores partially relieve the schema rigidity by allowing nested heterogeneous shapes. A purchase order can be a document with embedded items, addresses, and payment details, sparing the joins that relational schemas require. But the alphabet admits only a particular kind of structure: the labeled rooted tree. Domain semantic graphs are not constrained to trees — they admit cycles, references shared across documents, and edges of arbitrary type. When the same item is referenced from multiple orders, or when payments link to other documents, the missing edges must be fabricated at the application layer. The substrate handles trees natively and degrades the moment graph shape is required, in the strict graph-theoretic sense of §1.1.

Event sourcing, in its mainstream conception, improves on both by recording operations rather than state: replay reconstructs state by re-applying events. But in its conventional form — where events are serialized data structures mirroring the input commands — the entire input payload is captured regardless of which inputs the operation actually consumed. Consider an operation `RecordPayment` whose interface accepts twenty parameters; internally, the state-changing logic consumes four. The conventional implementation persists all twenty in the resulting event. The event payload is a fixed-shape tuple; the unconsumed fields constitute, in the information-theoretic sense of §1.1, structural noise alongside the consumed signal. Event sourcing inherits the porosity it was intended to escape: the journal records what the operation *received*, not what it *consumed*.

The persistence layer thus exhibits porosity in three forms, each shaped by its substrate's structural alphabet: state without operations in relational stores, fabricated edges where graph shape exceeds tree shape in document stores, and signal-mixed-with-noise in conventionally serialized event sourcing. The defect is the same — a representational alphabet that fails to match the structure being recorded. Three manifestations across three layers, one defect: §3 establishes the principle that addresses them as one.

3. The unified principle: anti-porosity

A system is anti-porous when its representations, at every layer, encode exactly the information consumed by the operations they describe — neither padding (which is sparsity) nor omission (which is loss). Anti-porosity is, in the precise vocabulary established in §1.1, the principled rejection of representational sparsity across all surfaces a system exposes — domain, endpoint, persistence — rather than at any one of them in isolation. In the language of type theory, this means representing heterogeneity as sum types rather than product types; in the language of information theory, it means a representation whose structural capacity matches its informational content.

The principle manifests as three simultaneous conditions, each inverting the manifestation observed in §2. First, the domain layer admits sum-type structure natively: a state hierarchy is encoded as variants, not as a product type with a `status` discriminator and conditionally populated columns. Second, the endpoint layer admits per-call selectivity at the wire: contracts express sum-type discrimination across heterogeneous use groups, not a fixed-shape DTO that unions all of them. Third, the persistence layer records operations and only the inputs the operations actually consumed: the journal carries signal, not the structural noise of every received-but-unused field.

The three conditions are not independent fixes; they are the consequence of a single decision applied at three surfaces. As §2 made structurally evident, each manifestation of porosity arises from the same architectural choice — to encode heterogeneous semantic content as product types, fixed-shape tuples whose excess capacity is paid in sparsity. Anti-porosity is the inverse choice — to encode heterogeneity as sum-type variants — applied as a single discipline across the three layers. A patch applied only at the domain (aggressive denormalization, hand-rolled inheritance) leaves the API and journal demanding fixed shapes; a patch applied only at persistence (compact event encoding) leaves the journal's density at the mercy of upstream layers' choices. Anti-porosity is not the sum of three local choices; it is a single architectural decision projected to three surfaces.

Operationally, anti-porosity is *density preservation*: the representation's structural capacity matches its informational content — signal retained, structural noise discarded. The combination of CQRS, the Actor Model, and Event Sourcing — conventionally presented as three separations of concern — admits a different reading once anti-porosity is named as the principle they jointly satisfy: they constitute, when applied as a single discipline, a mechanism for density preservation across representations. The principle does not prescribe a particular framework. Akka, Microsoft Orleans, Proto.Actor, or any system that combines actors, CQRS, and event sourcing could in principle satisfy the three conditions; doing so requires a substrate decision none of those frameworks has currently made (§7). The next section describes one realization — the Puppeteer framework — selected as the existence proof for the principle, not as its definition. The realization rests on a single architectural choice: that operations themselves — verbs invoked by callers — be recorded as the durable representation. This extends the principle of *code-as-data* (familiar from Lisp at the language layer) to the persistence layer, a property §4.3 develops formally as *homoiconic persistence*.

4. An instantiation: how Puppeteer realizes the three conditions

This section describes how the Puppeteer framework realizes the three conditions stated in §3. It is the only section of this paper in which authorial voice ("we") and the specifics of one implementation are concentrated. The realiza-

tion is presented as an existence proof for the principle, not as its definition; the conditions could be realized differently in another framework that made comparable substrate decisions (§7).

4.0 Origins of the instantiation

A practical encounter with the underlying phenomenon predates the present analysis by nearly two decades. In 2005, work that eventually became the Puppeteer framework began as a wrapper around a DLL that exposed services in the then-emerging vocabulary of *web services*. While building this wrapper, an unexpected property of the construction emerged. By intercepting the parameters of every incoming call and writing them, in order, to a log — what would now be called a *journal* — the full state of the underlying automaton could be recovered without inspecting its code. Starting from any consistent prior state and replaying the log, the automaton arrived at exactly the same final state. Persistence, traditionally treated as a separately designed concern, had emerged as a side effect of the wrapper. The phenomenon was named *autopersistence*, after its observable effect rather than from any theoretical premise. What the wrapper-log approach made visible was something the relational-persistence approach had hidden: the log was *dense* — it contained no empty fields, recording only what each call had actually used. The relational schemas the same applications wrote to, by contrast, were full of NULLs, columns that changed meaning by row, and joins that returned mostly-empty cells. The journal was *dense*; the schema was *porous*. The label and the principle followed; the observation came first. The mechanisms described in §4.1–§4.3 are the present-day form of properties already implicit in the 2005 wrapper: a domain visible to the framework only by reflection, parameters captured exactly as the call carried them, and a journal that records operations rather than state. Between the 2005 prototype and the present design, the framework was used in a sequence of internal projects, ported from Java to .NET, and progressively refined; the form described in §4.1–§4.3 has been in continuous production use since 2018.

4.1 Roles, not concepts: the library

Anti-porosity in the domain layer admits a single thesis: **a sum type, not a table**. The framework partitions the domain by *roles* — operations that act — rather than by *entities* — nouns that exist. Each role is a subtype: a class with its own fields, invariants, and verbs. The purchase order example of §2.1 is encoded directly as a sum type, with `Draft`, `Requested`, `Paid`, and `Dispatched` realized as distinct subtypes of a common base, each carrying only the attributes its role admits. The system's domain is the union of these roles, not a flattened table indexed by a `status` column.

The classes that realize these roles form what the framework calls the *library* — pure conceptual artifacts that do not know which "play" they participate in. A `Packer` is a *class puppet*: it knows how to pack; it does not know whether the packer instance is being persisted, replayed, or audited. The framework discov-

ers domain types reflectively at runtime, scanning the configured library assemblies (`Actor.cs:15` declares the marker base class; `DomainLibraries.cs:117-128` performs the assembly scan; `ActorHandler.cs:61` invokes it at handler construction). The domain need not register itself; the framework inspects what is there. The continuity is structural, not anecdotal: the 2005 wrapper (§4.0) intercepted parameters without inspecting the DLL it wrapped; reflection-based discovery preserves that property as a structural feature of the current design.

Polymorphism is a first-class concept of the DSL itself, distinct from how it is realized in the host language. Argument-parameter compatibility is computed at parse time via subtype checks (`AstExpression.cs:236-246`, `AreCompatible`); runtime substitution of subtypes is handled by the symbol table (`SymbolTable.cs:134`, `IsSubclassOf` check). Where the host language (C#) optimizes for verbose precision, the DSL optimizes for script readability and for boundary decoupling. Type promotion at the call site is permissive: polymorphic arrays promote to whatever collection type the library declares (`List`, `IEnumerable`, array, and so on); parameters accept any enumeration and promote at parse time to the specific enum the library expects. The DSL writes `Credit`, the DTO carries `Credit` as text, and the library's `PaymentMethod` enum receives the matching constant — no lexical change at any boundary. The contract is structurally decoupled from the domain class: each side may evolve its enum vocabulary independently, with the DSL mediating. **This is not syntactic sugar but a design decision: the DSL is shaped to read as domain narrative, not as a transcription of a programming language.**

Anti-porosity in the domain layer follows from the alignment of three properties: role-oriented partitioning (sum-type variants in the type hierarchy), reflection-based discovery (the framework imposes no schema on the library), and DSL-level polymorphism (sum-type discrimination is honored end-to-end in the operation contract). The remaining two layers — endpoint and persistence — are realized by separate but compatible mechanisms, treated in §4.2 and §4.3.

4.2 Parameter modifiers: `?` and `Eval`

The framework makes the direction of every value flow explicit through four parameter modifiers: `In`, `Out`, `InOut`, and `Eval`. The first three mirror the in/out/ref convention familiar from systems languages, ensuring caller-puppet isolation: the puppet cannot modify the caller's environment outside the declared direction. The fourth, `Eval`, is unique to this realization. The modifier system serves two complementary purposes: explicit isolation at the call contract, and honesty in the operation's journal record.

For `Out` parameters, the journal writes the literal character `?` as a placeholder (`Parameters.cs:755-757`). The reason is structural: an `Out` parameter's value is computed by the puppet, not supplied by the caller — there is no input value to record. At replay, the deserializer reads `?` and produces a default value of the

parameter's type (`Parameters.cs:780-782`); the puppet's logic re-executes and fills the slot. The journal thus records exactly what the call carried at the input boundary — output slots appear as honest reservations, not fictitious inputs.

`Eval` parameters address a different problem: values the puppet uses that are not supplied by the caller and are not deterministic across replays. Generated identifiers and game-session state — a random number or a gameboard captured for a game's session — are typical cases: values that pertain to the operation's semantics but cannot be re-derived at replay. The `Eval` modifier directs the puppet to capture the value, on first execution, as a literal-assigning script (`name = (type)(value);`), persisted as part of the operation's journal record (`Parameter.cs:33-36, 163-224, 228-247`). On replay, the script re-executes and assigns the same literal — determinism restored. V1's interpreted DSL had an `eval` command for the same purpose; V2 replaced it with the parameter modifier because the command form could not always be compiled (`EvalStatement.cs:52-55`).

The modifier system ensures the journal carries exactly what the call was:

- Caller inputs recorded as values.
- Puppet outputs recorded as `?` reservations.
- Non-deterministic values recorded as literal-capturing scripts.

Anti-porosity at the parameter level emerges from two complementary disciplines:

- Explicit direction at the call boundary.
- Explicit capture of non-determinism.

`?` and `Eval` are dual mechanisms that operationalize anti-porosity at parameter granularity. Integration into the broader journal — and the homoiconic property that allows replay against an evolved domain — is treated in §4.3.

4.3 Operations, not state: the homoiconic journal

In the framing developed here, the primary property of event sourcing is not temporal traceability but density preservation: it preserves semantic density that state-based persistence models discard.

By storing operations rather than state projections, the journal maintains an isomorphic representation of the semantic graph that tabular or document projections render sparse. The mechanism that achieves this is *homoiconic persistence*: each entry in the journal is simultaneously data — storable, indexable bytes — and a program — a parsable, executable script. The principle of homoiconicity originates in Lisp (McCarthy, 1960; Mooers, 1965) and was extended in the Smalltalk tradition (Kay); the framework extends it from the language layer to the persistence layer. The journal does not store a serialization of the actor's state; it stores the script that produced it. **Conventional event sourcing**

persists the command DTO; the present realization persists the execution script. These are not the same representational object. The former reproduces the representational sparsity diagnosed in §2.3; the latter cannot.

This homoiconic, isomorphic representation enables a capability that state-storing substrates structurally cannot offer: re-execution against new semantics. When the domain library evolves — a logic fix, a richer derivation, a new computation depending on past inputs — the journal replays against the updated library, producing corrected projections without altering the historical record. Storage engines that persist state, by contrast, can only restore the state that was written; the operations that produced it have been discarded with the noise.

The density preservation property is structural, not accidental. **A script cannot contain parameters the execution did not use, because the script is generated after parameter direction and evaluation rules have been applied** (§4.2). The journal therefore inherits density from the modifier discipline: caller inputs recorded as values, puppet outputs as `?` reservations, non-deterministic values as literal-capturing scripts. Two operational properties verify this in code. First, density is preserved through an asymmetry between inputs received and inputs consumed: only the latter are serialized (`Parameters.cs:755-757`, `:780-782`; `Lexer.cs:600`, where `?` is tokenised as `TokenType.question`). Second, the journal admits exactly two primitive operations — append and read-forward (`JournalWriter.cs:65`; `JournalReader.cs:35`) — exposed through a unifying interface (`Diary.cs`); the vocabulary of relational stores — SELECT, UPDATE, DELETE, INSERT, and the transactional locking that surrounds them — is absent by construction, not by convention.

A journal of DSL scripts therefore combines three properties — homoiconicity, graph preservation, and density preservation — under one representational substrate.

5. Empirical observations

The empirical claims in this paper are minimal by design. The contribution argued in §§1–4 is conceptual; quantitative measurements are deferred to a separate case-study paper. The principle predicts that representations satisfying the three conditions of §3 will exhibit density several orders of magnitude higher than relational projections of the same operations; the worked example below confirms the prediction on a generic case, and the deployed instantiation listed at the end of the section confirms it survives in production.

A worked example. Consider a generic domain in which buyers issue purchase orders against future scheduled events; an event is *confirmed* once it has occurred; and an event is *settled* with one of several outcomes (award, refund, restatement). The pattern recurs across ticketing, conditional pre-orders, and milestone escrow. Three primitive verbs realize the lifecycle:

Phase	Statement	Scope
Pre-event (× n)	`order = Company.Purchase(buyer, items, events, amount, currency)`	one order per call; ranges over items × events
Event occurs	`event.Confirm(date, operator)`	one event; implicitly all order-items bound to it
Resolution	`event.Settle(outcome, date, operator)`	one event; implicitly all order-items bound to it

The receiver determines scope; no row enumeration is required.

The journal model. The framework's V2 pattern stores each action's verb body once in an action library; the journal records per invocation only the action identifier, the parameter values, and minimal administrative metadata (timestamp, operator, entry ID). The journal grows with parameters, not with action bodies. (V1's raw-script representation, in which the verbatim script appears in each entry, is preserved for backward compatibility.) This split mirrors the separation between definition and invocation in any compiled or interpreted language; in this realization, that separation extends to persistence.

Storage backends. The conditions of §3 concern the representational shape of journal entries, not the substrate that physically stores them. Four backends ship with the framework — filesystem (UTF-8 script entries in append-only files), in-memory (for testing and ephemeral actors), and two relational engines (MySQL and SQL Server) — and the homoiconicity, density, and append-only access patterns are preserved across all four. The relational backends expose the journal through standard SQL interfaces familiar to operations teams; entries remain executable scripts queryable as text columns. Choice of backend is a deployment decision; the structural properties the paper attributes to the journal are invariant across that choice. A commonly raised concern — that a journal is a black box for operations staff who must inspect it under pressure — is therefore largely a deployment-configuration question rather than a property of the principle.

The structural gap. A `Confirm` invocation in the journal carries the parameters of one verb plus its administrative metadata — typically tens of bytes. The same operation expressed as relational mutations must enumerate every order-item bound to the event, copying parent dimensions onto every child row, because SQL's `JOIN` cannot quantify — it materializes Cartesian projections of existing rows, it does not abbreviate them. For an event with many bound items, the relational expansion exceeds the script representation by several orders of magnitude. The constant depends on aggregate cardinality; the existence of the gap does not.

Why the gap is structural. The compactness of the script representation

rests on three model properties:

- **Quantification.** The receiver implicitly ranges over the affected aggregate; the verb applies to all members at once.
- **Polymorphism.** A single `Settle(outcome, ...)` accepts heterogeneous outcomes as a sum type, sharing representation across branches.
- **Parameters as metadata.** Operator, date, and outcome travel as arguments of one statement; in the relational form they are copied into every affected row, because the row is the unit of identity.

The relational form has none of these. A `JOIN` cannot quantify universally; an `UPDATE … WHERE …` issues the directive, but its result is enumerated rows. The porosity of the relational layer denotes here, in concrete terms, the structural consequence of substituting *“ item event”* with *“N copies of the antecedent.”*

Forensic observation. The structural gap is not specific to this example. Any mature relational schema can be read for porosity directly: counting a verb's parameters against the columns of the rows its invocation affects, identifying NULL-allowed fields whose values the originating operation never had to supply, and noting cross-product tables that materialize what the verb expressed as quantification. The procedure is reproducible across public corpora — open-source schemas in any domain — and its result is robustly the same: representations exceeding their minimal generators by several orders of magnitude. The worked example demonstrates the property; it does not constitute its only evidence. A companion repository alongside this paper hosts worked applications of this procedure and small-scale demonstrations of the framework's mechanisms, open to review and contribution.

Existence proof in production. A system satisfying the three conditions of §3 has been in continuous production use since 2018, after more than a decade of preceding refinement (described in §4.0) traceable to a 2005 prototype. Puppeteer, the framework described in §4, currently runs structurally distinct subsystems within the core of eCommerce and payments platforms: payment hubs, account-balance ledgers, KYC pipelines, customer-facing storefronts and experiences, and payment-processor integrations. Code references throughout this paper (Appendix A) point to verifiable mechanism implementations. A separate case-study paper presents quantitative observations from these deployments — endpoint latency distributions, journal growth rates, replay performance, and developer-velocity comparisons — alongside the operational details (deployment, workload, infrastructure) that fall outside the scope of a conceptual paper. Those measurements support the conceptual claim without constituting its core: the principle would not become false if the deployment were withdrawn, only less obviously confirmed.

6. Counter-arguments

This section addresses the strongest objections to the argument advanced in §§1–5. Each objection is presented in its strongest form before its rebuttal; rebuttals

draw on the principle articulated in §§1–3, not on the specific realization of §4.

"This is just CQRS done well." CQRS already separates read and write; what does anti-porosity add? Two things, addressed in Claim 5 and §3. First, CQRS is one of four traditions that document a face of porosity locally; anti-porosity is the unified rejection of porosity across all four. CQRS implementations vary widely in their porosity properties — some recreate porous DTOs and porous event payloads despite separating reads from writes. Second, the contribution of this paper is not any pattern (CQRS or otherwise) but the recognition that the four traditions converge on a single defect, which prior work has not named with sufficient generality to see. **CQRS addresses separation of concerns; anti-porosity addresses representational form. The two operate at different conceptual layers.**

"In small systems, porosity doesn't hurt." True. The operational costs of porosity scale with system size, longevity, and rate of change. The conceptual claim, however, does not depend on these. Porosity is a structural property visible at any scale; whether one chooses to address it depends on the regime in which the system operates. This paper does not advocate adopting any specific framework for small or short-lived systems; *When NOT to use this approach* explicitly enumerates regimes where the realization pattern is not the right fit, and the principle itself remains diagnostic in those regimes — porosity remains a real pathology, even where its correction would not repay its cost. **The claim of this paper is therefore diagnostic rather than prescriptive: it names a defect whose relevance depends on scale, not one whose correction is universally mandatory.**

"Joins are sometimes necessary." Anti-porosity does not prohibit joins. **As illustrated in §5, joins migrate to the read side, where enumeration serves analytical needs rather than polluting the representation of operations.** The unified principle (§3) constrains only how state is *recorded* — not how it is queried for derivative purposes. Read projections may be denormalized, joined, indexed, and aggregated however a downstream consumer requires; the journal remains dense, but analytical and reporting use cases retain the full vocabulary of relational queries against derived views.

"Business intelligence and analytics require SQL." Conventional BI assumes that the relational store *is* the source of truth. Under anti-porosity, the operation log is the source of truth, and analytical projections are downstream consumers. SQL access remains available — to the projection, not to the source of truth. **Because the operation log stores operations rather than state (§3, §4.3), projections for BI can be recomputed with evolving semantics — a capability unavailable when the relational store is itself the historical record.**

"Our data lake is the source of truth." This is an organizational decision, not a structural property of the system. The data lake's role can be reframed: it remains a shared projection consumed by downstream teams, while the opera-

tion log is the source of truth for the producer system. Anti-porosity is preserved internally even when external contracts demand denormalized projections at the system boundary. **The distinction is between organizational source of truth and architectural source of truth. Anti-porosity concerns the latter.**

7. Related work

This paper sits at the intersection of multiple bodies of work that, until now, have addressed faces of porosity locally. Each is reviewed below not for completeness but for the specific position the present paper occupies relative to it.

Four applied traditions. *Domain-driven design* (Evans, 2003; Vernon, 2013) documents *anemic domain models* and *persistence-driven design* as pathologies and prescribes ubiquitous language and aggregate-rooted modeling as remedies. It does not name the underlying defect as representational projection; it treats it as artifact of design discipline. *REST API design* (Fielding, 2000; Richardson and Ruby, 2008) and the GraphQL specification (2015) document overloaded endpoints and fixed-shape DTOs, prescribing resource orientation and field selection respectively. GraphQL in particular comes closest to anti-porosity at the endpoint layer: its field selection is, in effect, a sum-type projection at the wire — but the discipline lacks corresponding sum-type expression in the domain or persistence layers. *Event Sourcing* (Young, 2010; Fowler, 2005; Vernon, 2013) documents event-payload bloat and command-event coupling and prescribes command-event separation, upcasting, and versioning; it recognizes that persisting state is insufficient and persists operations instead, but typically represents those operations as fixed-shape data structures whose capacity exceeds the information the execution consumed. *Database theory* (Codd, 1970; Kent, 1983; Fagin, 1977) is the oldest and most formal of the four; it documents normalization anomalies and NULL semantics and prescribes successive normal forms. It names the effects (anomalies) more than the cause (representational sparsity from substrate-driven projection).

These traditions have rarely engaged with each other on the question of representational form, perhaps because their domains appear to address different problems. This paper argues that they are observing the same structural defect through different lenses. Each tradition names a local defect and prescribes a local remedy. None observes the underlying common cause: a substrate whose structural alphabet exceeds the operations the representation must record.

Four formalisms. The grounding established in §1.1 invokes existing formal vocabulary. Graph theory (Diestel, 2017), with extension to typed and labeled graphs in the Semantic Web (Berners-Lee, Hendler, and Lassila, 2001) and graph databases (Robinson, Webber, and Eifrem, 2015), provides the language of semantic graphs. Information theory (Shannon, 1948) provides structural ca-

pacity, informational content, and signal-versus-noise. Storage-engine literature (Hellerstein, Stonebraker, and Hamilton, 2007) provides the language of structural alphabets exposed by substrates. Database normalization theory (Codd, 1970, 1971; Fagin, 1977) provides the canonical historical formalism for the relational case. The paper does not invent formalism; the contribution is the observation that these four already-established formalisms agree on a single defect that none names with sufficient generality across substrates. **The present paper's formal vocabulary is therefore assembled, not invented.**

Aspect-oriented programming and ORM annotations. The aspect-oriented programming tradition (Kiczales et al., 1997; AspectJ; Spring AOP) identified persistence as one of the ugliest cross-cutting concerns and sought to externalize it from domain code. The most widespread practical instantiation is ORM-style annotations (the Java Persistence API, Hibernate, Entity Framework). These approaches share a goal with anti-porosity — keeping the domain class free of persistence concerns — but operate at a different level: they decorate the domain class with framework-specific metadata that describes how to map its fields to a relational substrate. The substrate's structural alphabet is preserved; only the syntactic appearance of pollution is moved from the class body to its annotations. Anti-porosity, by contrast, addresses the substrate itself: by replacing relational projections with a homoiconic journal of operations (§4.3), the question of how to map domain fields to relational columns disappears. **The domain class need not be annotated, because the persistence substrate no longer requires a projection of its fields.**

Other actor-model frameworks. Multiple frameworks implement combinations of CQRS, the Actor Model, and Event Sourcing. Akka (Lightbend) on the JVM, Microsoft Orleans (Bernstein et al., 2014) with its grain abstraction, Proto.Actor across multiple languages, and the dedicated EventStore database all share substantial conceptual infrastructure with the realization presented in §4. They differ from it, and from one another, in a single consistent respect: **each persists events as serialized data structures defined at the command boundary, rather than as executable scripts derived after parameter evaluation**. The underlying storage organization typically retains relational patterns — category-indexed streams, table-backed projections — rather than per-actor append-only structures. None currently articulates anti-porosity as a unified principle, and none persists scripts in the homoiconic sense developed in §4.3. The differences are not failures of these frameworks against their stated goals — each does what it sets out to do — but they illustrate that anti-porosity requires a deliberate decision at the substrate layer that mainstream actor frameworks have not made. The decision is available to them: any of these frameworks could be extended to record operations as scripts rather than as DTOs, and to migrate the storage organization from category-indexed streams to per-actor append-only journals. **The distinction is not in the use of actors, CQRS, or event sourcing, but in what is chosen as the durable representational unit.**

What this paper adds. The paper does not propose a new pattern; it identifies that four applied traditions converge on a single defect, names the defect (porosity) and its rejection (anti-porosity), and demonstrates that the rejection is implementable as a unified discipline. The Puppeteer framework serves as proof-of-existence; the conceptual claim is independent of any specific framework, and an open invitation is extended to other realizations that satisfy the same conditions. **Its novelty lies in the act of synthesis and in showing that the synthesis is operationally realizable.**

8. Conclusion

This paper has identified a structural defect — *porosity* — that four bodies of work have documented locally without naming as a single phenomenon. The defect arises whenever a representation's structural alphabet exceeds the operations the representation must record, and its principled rejection — *anti-porosity* — requires three simultaneous conditions: a domain whose hierarchy admits sum-type structure natively, an endpoint whose contract permits per-call selectivity, and a persistence layer that records operations rather than state. A system that satisfies all three through a single architectural decision — the homoiconic journal — has been in continuous production use since 2018, after more than a decade of preceding refinement traceable to a 2005 prototype (§4.0); that system, Puppeteer, is presented in §4 as the existence proof for the principle, not as its definition. The conceptual claim is independent of any specific framework, and other systems that make comparable substrate decisions could realize it equivalently. In the vocabulary established in §1.1, anti-porosity aligns structural capacity with informational content across the three representational layers a system exposes.

The implications extend beyond the paper. When porosity is removed at the substrate, much of the infrastructure traditionally introduced to compensate for substrate mismatch becomes optional rather than essential: caches that hid slow joins, ORMs that translated between domain and table, message queues that moved flat tuples between systems. The question of how systems built on porous substrates may adopt the new discipline progressively follows from the autopersistence property the framework inherits from its 2005 origin: a system whose initial state and event sequence are observable can be reconstructed independently of how it persists internally; a system whose only durable artifact is state cannot reconstruct the operations that produced it.

Several lines of work follow. A separate case-study paper will report quantitative measurements from the framework's production use. Four companion papers extend the present argument: one on dual compilation as the strategy that makes the homoiconic journal both interpretable and compilable; one on Reactions and the consistency partition the framework admits; one on zero-downtime deployment as a corollary of journal-based state handoff; and one on the broader DBMS-centric ecosystem, treating its compensating infrastructure as symptoms whose necessity dissolves once the substrate is changed. Inde-

pendently, the forensic procedure introduced in §5 invites verification against public corpora — an exercise any reader can carry out on schemas in their own domain.

The contribution of this paper is naming. The work that follows is to show that the name corresponds to a discipline that can be sustained, measured, and extended across the substrate decisions that have shaped software for half a century. Naming the construct allows porosity to be identified in contexts where it had previously been absorbed as a routine cost of software construction.

Cross-references

This paper establishes vocabulary that subsequent papers in the series reuse:

- *Paper 2 — Dual compilation* (published v0.1-draft): the anti-porosity principle as it manifests at the language layer (interpreted versus compiled execution paths).
- *Paper 3 — Reactions and the partition: opt-in eventual consistency in actor-native systems* (published v0.1-draft): the consistency model that anti-porosity admits, with deferred derivative behaviors expressed as domain code rather than externalized pipelines.
- *Paper 4 — Zero-downtime deployment via journal-based state handoff* (forthcoming): the journal's density is prerequisite for the skip mechanism and red-black deployment patterns.
- *Paper 5 — Why Redis is a symptom* (forthcoming): extends the anti-porosity argument to the broader infrastructure ecosystem, treating compensating layers (caches, ORMs, message queues) as artifacts whose necessity dissolves once the substrate is changed.

Code provenance

Source-code references in this paper resolve against the public Puppeteer repository at commit `2f31f96` (2026-05-18). The snapshot is archived in Software Heritage under the following persistent identifier:

```
swh:1:dir:10e7e6bad7eb77b6c2e406762026177f95c3ae92;
  origin=https://github.com/alvaroNCubo/puppeteer;
  anchor=swh:1:rev:2f31f9674a5de816bdf1bf9d8360ff218a02e4da
```

Inline references of the form `file.cs:NN` (e.g., `ActorHandler.cs:38`) resolve against this snapshot. A reader can construct a per-file SWHID by adding the qualifiers `;path=<path>;lines=<NN>` to the directory SWHID above. Future commits to the repository may renumber lines; the SWHID preserves the cited state independently of any future change to the repository or its hosting.

Acknowledgments

The author used large language models (including Claude and ChatGPT) as editorial assistants for language refinement, structural feedback, and literature navigation. All original ideas, terminology, theoretical constructs, and technical content presented in this work are solely the author's.

References

Berners-Lee, T., Hendler, J., & Lassila, O. (2001). The Semantic Web. *Scientific American*, 284(5), 34–43.

Bernstein, P., Bykov, S., Geller, A., Kliot, G., & Thelin, J. (2014). *Orleans: Distributed virtual actors for programmability and scalability* (Microsoft Research Technical Report MSR-TR-2014-41). Microsoft Research.

Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387.

Codd, E. F. (1971). *Further normalization of the data base relational model* (IBM Research Report RJ909). IBM.

Diestel, R. (2017). *Graph theory* (5th ed.). Springer.

Evans, E. (2003). *Domain-driven design: Tackling complexity in the heart of software*. Addison-Wesley.

Fagin, R. (1977). Multivalued dependencies and a new normal form for relational databases. *ACM Transactions on Database Systems*, 2(3), 262–278.

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* (Doctoral dissertation, University of California, Irvine).

Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.

Fowler, M. (2005). *Event sourcing*. martinfowler.com. https://martinfowler.com/eaaDev/EventSourcing.html

GraphQL Foundation. (2015). *GraphQL specification*. https://spec.graphql.org

Hellerstein, J. M., Stonebraker, M., & Hamilton, J. (2007). Architecture of a database system. *Foundations and Trends in Databases*, 1(2), 141–259.

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105.

Kay, A. C. (1993). The early history of Smalltalk. *ACM SIGPLAN Notices*, 28(3), 69–95.

Kent, W. (1983). A simple guide to five normal forms in relational database theory. *Communications of the ACM*, 26(2), 120–125.

Kiczales, G., Lamping, J., Mendhekar, A., Maeda, C., Lopes, C., Loingtier, J.-M., & Irwin, J. (1997). Aspect-oriented programming. In M. Akşit & S. Matsuoka (Eds.), *ECOOP'97 — Object-oriented programming* (pp. 220–242). Springer.

McCarthy, J. (1960). Recursive functions of symbolic expressions and their computation by machine, Part I. *Communications of the ACM*, 3(4), 184–195.

Mooers, C. N. (1965). TRAC, a procedure-describing language for the reactive typewriter. In *Proceedings of the 20th National Conference of the ACM* (pp. 229–246).

Richardson, L., & Ruby, S. (2008). *RESTful web services*. O'Reilly Media.

Robinson, I., Webber, J., & Eifrem, E. (2015). *Graph databases* (2nd ed.). O'Reilly Media.

Shannon, C. E. (1948). A mathematical theory of communication. *Bell System Technical Journal*, 27(3), 379–423; 27(4), 623–656.

Vernon, V. (2013). *Implementing domain-driven design*. Addison-Wesley.

Young, G. (2010). *CQRS documents*. https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf

---

Appendix A: Source-code verification

This appendix consolidates the source-code references cited throughout the paper. All paths are relative to the public Puppeteer repository; line numbers reflect the version of the code at the time of writing. The framework's source was migrated from Spanish to English identifiers between v0.1 and v0.2 of this paper; the line numbers and file names below reflect the current English-language source.

A.1 Reflection-based discovery (§4.1)

Claim	File	Lines
`Actor` marker base class for domain entry points	`Puppeteer/Actor.cs`	15
Reflection-based discovery of domain types via assembly scan	`Puppeteer/EventSourcing/DomainLibraries.cs`	17–128
Domain library loaded at handler construction	`Puppeteer/EventSourcing/ActorHandler.cs`	67
Polymorphic argument-parameter compatibility at parse time (`AreCompatible`)	`Puppeteer/EventSourcing/Interpreter/Libraries/AstExpression.cs`	236–246

Claim	File	Lines
Runtime substitution of subtypes via symbol table (`IsSubclassOf` check)	`Puppeteer/EventSourcing/Interpreter/SymbolTable.cs`	134

A.2 Parameter modifiers (§4.2)

Claim	File	Lines
Four parameter modifiers defined (`In`, `Out`, `InOut`, `Eval`)	`Puppeteer/Parameter.cs`	33–36
`Eval` value setter generates literal-assigning script on first invocation	`Puppeteer/Parameter.cs`	163–224
`EvalScript` property (valid only for `Eval` parameters)	`Puppeteer/Parameter.cs`	228–247
Parser recognises `Eval` modifier in parameter declarations	`Puppeteer/Parameters.cs`	65, 110–111
`Eval` parameters' `EvalScript` serialized to journal	`Puppeteer/Parameters.cs`	334–367
`Out` parameters serialize as `?` placeholder	`Puppeteer/Parameters.cs`	755–757
`?` for `Out` parameters deserializes to default value	`Puppeteer/Parameters.cs`	780–782
`?` tokenised as `TokenType.question` in the lexer	`Puppeteer/EventSourcing/Interpreter/Lexer.cs`	600
V1 `eval` statement preserved (interpreted only); V2 redirects to parameter modifier	`Puppeteer/EventSourcing/Interpreter/Libraries/EvalStatement.cs`	52–55

A.3 Homoiconic journal (§4.3)

Claim	File	Lines
Journal high-level interface (append-style writes, read-forward only)	`Puppeteer/EventSourcing.EventJournal.cs`	228–257
Append primitive (`AppendRecord`)	`Puppeteer/EventSourcing.EventJournal/FileSystem/JournalWriter.cs`	65
Read-forward primitive (`ReadAll`)	`Puppeteer/EventSourcing.EventJournal/FileSystem/JournalReader.cs`	35
Script entries persisted as UTF-8 bytes (`EncodeScriptEvent`)	`Puppeteer/EventSourcing.EventJournal/FileSystem/BinaryEventCodec.cs`	101–102
`ReplayEvent` re-executes scripts at journal replay	`Puppeteer/EventSourcing.EventJournal/ActorHandler.cs`	84–86
On-demand parser invocation for type resolution at replay	`Puppeteer/EventSourcing.EventJournal/Reaction.cs`	1027–1029